

Assignment - 8

Course: MST501-1: Python
Programming for Statistics



Done by:

Name: Ankita Das

Register Number: 2648103

Date: 06/08/2026

1. A food delivery company wants to evaluate operational efficiency using a dataset containing restaurant ratings, preparation time, delivery distance, delivery time, weather conditions, order value, and customer feedback scores (download/create CSV up to sufficient rows. Using Python, analyse whether longer delivery distances always lead to delayed deliveries, identify restaurants with consistently high ratings despite long delivery times, and create suitable visualizations to compare delivery performance across weather conditions and order values. Conclude with recommendations that could improve customer satisfaction.

#1

"""

```
import pandas as pd
import matplotlib.pyplot as plt
df =
pd.read_csv(r"C:\\Users\\Rajku\\OneDrive\\Desktop\\Python\\food_delivery_dataset
(1).csv")
print("First 5 rows of the dataset")
print(df.head())
print(df.info())
print(df.describe())
correlation = df['delivery_distance_km'].corr(df['delivery_time_min'])
print("Correlation between delivery distance and delivery time")
plt.figure(figsize=(6,4))
plt.scatter(df['delivery_distance_km'], df['delivery_time_min'])
plt.title('Delivery Distance vs Delivery Time')
plt.xlabel('Delivery Distance (km)')
plt.ylabel('Delivery Time (minutes)')
plt.show()
average_time = df['delivery_time_min'].mean()
print("Average Delivery Time",average_time )
long_delivery = df[df['delivery_time_min'] > average_time]
restaurant_analysis = long_delivery.groupby('restaurant')[['restaurant_rating',
'delivery_time_min']].mean()
high_rated = restaurant_analysis[restaurant_analysis['restaurant_rating'] >= 4.4]

print("Restaurants with high ratings despite long delivery times")
print(high_rated)
weather_avg = df.groupby('weather')['delivery_time_min'].mean()
print("Average Delivery Time by Weather")
print(weather_avg)
plt.figure(figsize=(6,4))
weather_avg.plot(kind='bar')
plt.title('Average Delivery Time by Weather')
plt.xlabel('Weather')
```

```

plt.ylabel('Average Delivery Time (minutes)')
plt.show()
def category(value):
    if value < 300:
        return 'Low'
    elif value < 600:
        return 'Medium'
    elif value < 900:
        return 'High'
    else:
        return 'Premium'
df['order_category'] = df['order_value'].apply(category)
order_avg = df.groupby('order_category')['delivery_time_min'].mean()
print("Average Delivery Time by Order Value Category")
print(order_avg)
plt.figure(figsize=(6,4))
order_avg.plot(kind='bar')
plt.title('Average Delivery Time by Order Value Category')
plt.xlabel('Order Value Category')
plt.ylabel('Average Delivery Time (minutes)')
plt.show()
feedback_avg = df.groupby('restaurant')['customer_feedback_score'].mean()
print("Average Customer Feedback Score by Restaurant")
print(feedback_avg)
plt.figure(figsize=(8,4))
feedback_avg.plot(kind='bar')
plt.title("Average Customer Feedback Score by Restaurant")
plt.xlabel('Restaurant')
plt.ylabel('Average Feedback Score')
plt.show()
print("The analysis shows that delivery distance and delivery time have a positive relationship")
print("However, longer delivery distances do not always lead to delayed deliveries because weather conditions and preparation time also affect delivery time")
print("Some restaurants maintain high ratings even when delivery times are longer, indicating that food quality and service can keep customers satisfied")
print("Rain and storm conditions result in the highest delivery times, while order value has only a small effect on delivery performance")
print("Customer feedback scores help identify restaurants that provide a better overall customer experience")
print("Recommendations")
print("Increase delivery staff during rain and storm conditions")
print("Improve route planning for long-distance deliveries")
print("Reduce restaurant preparation time")

```

```
print("Provide accurate delivery time estimates and real-time tracking")
print("Offer discounts or coupons for delayed deliveries")
print("Learn from restaurants with high ratings and feedback scores to improve overall service quality")
```

```
#1
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv(r"C:\Users\Rajku\OneDrive\Desktop\Python\fitness_training_program_data\dataset.csv")
print("First 5 rows of the dataset")
print(df.head())
print(df.info())
print(df.describe())
correlation = df['delivery_distance_km'].corr(df['delivery_time_min'])
print("Correlation between delivery distance and delivery time")
plt.figure(figsize=(6,4))
plt.scatter(df['delivery_distance_km'], df['delivery_time_min'])
plt.title('Delivery Distance vs Delivery Time')
plt.xlabel('Delivery Distance (km)')
plt.ylabel('Delivery Time (minutes)')
plt.show()
average_time = df['delivery_time_min'].mean()
print("Average Delivery Time", average_time)
long_delivery = df[df['delivery_time_min'] > average_time]
restaurant_analysis = long_delivery.groupby('restaurant')[['order_id', 'restaurant_rating', 'prep_time_min', 'delivery_distance_km', 'delivery_time_min', 'weather', 'order_value', 'customer_feedback_score']]
high_rated = restaurant_analysis[restaurant_analysis['restaurant_rating'] > 4]
print("Restaurants with high ratings despite long delivery time")
print(high_rated)
weather_avg = df.groupby('weather')['delivery_time_min'].mean()
print("Average Delivery Time by Weather")
print(weather_avg)
plt.figure(figsize=(6,4))
weather_avg.plot(kind='bar')
plt.title('Average Delivery Time by Weather')
plt.xlabel('Weather')
plt.ylabel('Average Delivery Time (minutes)')
plt.show()
def category(value):
    if value < 300:
        return 'Low'
    elif value < 600:
        return 'Medium'
    elif value < 900:
        return 'High'
    else:
        return 'Other'
```

2 A

fitness centre introduced a new eight-week training program and collected participants' body weight before and after completing the program (download/create CSV up to sufficient rows).

The management

wants to determine whether the program significantly reduced body weight. Using SciPy, formulate the null and alternative hypotheses, select an appropriate statistical test, perform the hypothesis test at a 5% significance level, interpret the p-value, and conclude whether the program was effective. Explain why the chosen statistical test is appropriate for this scenario.

```
import pandas as pd
from scipy.stats import ttest_rel
df = pd.read_csv(r"C:\Users\Rajku\OneDrive\Desktop\Python\fitness_training_program_data\dataset.csv")
print("First 5 rows of the dataset")
print(df.head())
print("Null Hypothesis (H0): The training program did not reduce body weight")
print("Alternative Hypothesis (H1): The training program reduced body weight")
t_stat, p_value = ttest_rel(df['body_weight_before'], df['body_weight_after'])
```

```

df['weight_before_kg'],
df['weight_after_kg']
)
print("T-statistic", t_stat)
print("P-value:", p_value)
alpha = 0.05
if p_value < alpha:
    print("Decision: Reject the null hypothesis (H0)")
    print("Conclusion: The 8-week training program significantly reduced body weight")
else:
    print("Decision: Fail to reject the null hypothesis (H0)")
    print("Conclusion is- The program did not significantly reduce body weight")
before_mean = df['weight_before_kg'].mean()
after_mean = df['weight_after_kg'].mean()
print("Average weight before program in kg", before_mean)
print("Average weight after program in kg", after_mean)
print("Reason for using paired t-test:")
print("The same participants were measured before and after the training program.")
print("Therefore, the observations are related (paired), and the test compares the average weight difference.")

```

```

#<
import pandas as pd
from scipy.stats import ttest_rel
df = pd.read_csv(r"C:\Users\Rajku\OneDrive\Desktop\Python\file1.csv")
print("First 5 rows of the dataset")
print(df.head())
print("Null Hypothesis (H0): The training program did not reduce body weight")
print("Alternative Hypothesis (H1): The training program reduced body weight")
t_stat, p_value = ttest_rel(df['weight_before_kg'], df['weight_after_kg'])
print("T-statistic", t_stat)
print("P-value:", p_value)
alpha = 0.05
if p_value < alpha:
    print("Decision: Reject the null hypothesis (H0)")
    print("Conclusion: The 8-week training program significantly reduced body weight")
else:
    print("Decision: Fail to reject the null hypothesis (H0)")
    print("Conclusion is- The program did not significantly reduce body weight")
before_mean = df['weight_before_kg'].mean()
after_mean = df['weight_after_kg'].mean()
print("Average weight before program in kg", before_mean)
print("Average weight after program in kg", after_mean)
print("Reason for using paired t-test:")
print("The same participants were measured before and after the training program.")
print("Therefore, the observations are related (paired), and the test compares the average weight difference.")

```

```

Average weight after program in kg 72.58999999999999
Reason for using paired t-test:
The same participants were measured before and after the training program.
Therefore, the observations are related (paired), and the test compares the average weight difference.
Index(['Vehicle_ID', 'Battery_Temperature (°C)', 'Charging_Time (hrs)', 'Charging_Cycles', 'Driving_Distance (km)', 'Energy_Consumption (kWh/100km)', 'Battery_Health (%)', 'Charging_Type'],
      dtype='str')
First 5 rows:
  Vehicle_ID  Battery_Temperature (°C)  ...  Battery_Health (%)  Charging_Type
0  Vehicle_ID                        32.0  ...                96.0        Normal
1  Vehicle_ID                        45.0  ...                84.0          Fast
2  Vehicle_ID                        30.0  ...                97.0        Normal
3  Vehicle_ID                        48.0  ...                80.0          Fast
4  Vehicle_ID                        35.0  ...                92.0        Normal

[5 rows x 8 columns]
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Vehicle_ID                            20 non-null    str
1   Battery_Temperature (°C)              19 non-null    float64
2   Charging_Time (hrs)                   19 non-null    float64

```

3 An

electric vehicle manufacturer has collected data on battery temperature, charging time, charging cycles, driving distance, energy consumption, and battery health percentage for thousands of vehicles (download/create CSV up to sufficient rows).

Using Python,

first, perform data cleaning and exploratory analysis with Pandas, Matplotlib, and Seaborn to identify trends and anomalies. Next, test whether fast charging significantly reduces battery health using an appropriate hypothesis test in SciPy. Finally, build a regression model to predict battery health based on the

available features, evaluate the model, and provide recommendations that could help increase battery lifespan.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from scipy.stats import ttest_ind
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv("EV_Battery_Data.csv")
```

```
print("First 5 Rows")
print(df.head())
```

```
print("\nDataset Information")
print(df.info())
```

```
print("\nStatistical Summary")
print(df.describe())
```

```
print("\nMissing Values")
print(df.isnull().sum())
```

```
df.drop_duplicates(inplace=True)
```

```
numeric_columns = df.select_dtypes(include=np.number).columns
```

```
for col in numeric_columns:
    df[col].fillna(df[col].mean(), inplace=True)
```

```
print("\nMissing Values After Cleaning")
print(df.isnull().sum())
```

```
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
```

```
plt.show()
```

```
plt.figure(figsize=(6,4))  
sns.scatterplot(x='Battery_Temperature', y='Battery_Health', data=df)  
plt.title("Battery Temperature vs Battery Health")  
plt.show()
```

```
plt.figure(figsize=(6,4))  
sns.scatterplot(x='Charging_Cycles', y='Battery_Health', data=df)  
plt.title("Charging Cycles vs Battery Health")  
plt.show()
```

```
plt.figure(figsize=(6,4))  
sns.histplot(df['Battery_Health'], bins=10, kde=True)  
plt.title("Battery Health Distribution")  
plt.show()
```

```
plt.figure(figsize=(6,4))  
sns.boxplot(y=df['Battery_Health'])  
plt.title("Battery Health Boxplot")  
plt.show()
```

```
plt.figure(figsize=(5,4))  
sns.countplot(x='Charging_Type', data=df)  
plt.title("Charging Type")  
plt.show()
```

```
fast = df[df["Charging_Type"]=="Fast"]["Battery_Health"]  
normal = df[df["Charging_Type"]=="Normal"]["Battery_Health"]
```

```
t_stat, p_value = ttest_ind(fast, normal)
```

```
print("\nIndependent t-Test")  
print("T-statistic =", round(t_stat,4))  
print("P-value =", round(p_value,4))
```

```
alpha = 0.05
```

```
if p_value < alpha:
    print("\nReject Null Hypothesis")
    print("Fast charging significantly affects battery health.")
else:
    print("\nFail to Reject Null Hypothesis")
    print("Fast charging does not significantly affect battery health.")
```

```
X = df[['Battery_Temperature',
        'Charging_Time',
        'Charging_Cycles',
        'Driving_Distance',
        'Energy_Consumption']]
```

```
y = df['Battery_Health']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print("\nRegression Model Evaluation")
print("Mean Squared Error :", round(mse,2))
print("R2 Score :", round(r2,2))
```

```
plt.figure(figsize=(7,5))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Battery Health")
plt.ylabel("Predicted Battery Health")
```



```
plt.title("Actual vs Predicted Battery Health")
plt.show()
```

```
coefficients = pd.DataFrame({
    'Feature':X.columns,
    'Coefficient':model.coef_
})
```

```
print("\nRegression Coefficients")
print(coefficients)
```

```
print("\nObservations")
```

```
print("1. Correlation matrix shows relationships among variables.")
print("2. Scatter plots show how battery health changes with temperature and
charging cycles.")
print("3. Boxplot helps identify anomalies (outliers).")
print("4. Histogram shows the distribution of battery health.")
print("5. T-test determines whether fast charging significantly reduces battery
health.")
print("6. Regression predicts battery health using the available features.")
```

```
print("\nRecommendations")
```

```
print("- Avoid frequent fast charging whenever possible.")
print("- Keep battery temperature under control.")
print("- Reduce unnecessary charging cycles.")
print("- Follow manufacturer-recommended charging practices.")
print("- Monitor battery health regularly.")
print("- Drive efficiently to reduce energy consumption.")
```

```
file_path = r"C:\Users\Rajku\OneDrive\Desktop\Python\questio
df = pd.read_csv(file_path)
print("First 5 rows:")
print(df.head())
print("Dataset Information:")
print(df.info())
print("Statistical Summary:")
print(df.describe())
df.rename(columns={
    "Battery_Temperature (°C)": "Battery Temperature",
    "Charging_Time (hrs)": "Charging Time",
    "Charging_Cycles": "Charging Cycles",
    "Driving_Distance (km)": "Driving Distance",
    "Energy_Consumption (kWh/100km)": "Energy Consumption",
    "Battery_Health (%)": "Battery Health Percentage",
    "Charging_Type": "Charging Type"
}, inplace=True)
print("Updated Columns:")
print(df.columns)
print("Missing Values:")
print(df.isnull().sum())
df.drop_duplicates(inplace=True)
for col in df.columns:
    if df[col].dtype in ["float64", "int64"]:
        df[col].fillna(
            df[col].median(),
            inplace=True
        )
    else:
        df[col].fillna(
            df[col].mode()[0],
            inplace=True
        )
print("After Cleaning:")
print(df.isnull().sum())
plt.figure(figsize=(10,6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)
```

[5 rows x 8 columns]
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 8 columns):
Column Non-Null Count Dtype

0 Vehicle_ID 20 non-null str
1 Battery_Temperature (°C) 19 non-null float64
2 Charging_Time (hrs) 19 non-null float64
3 Charging_Cycles 19 non-null float64
4 Driving_Distance (km) 19 non-null float64
5 Energy_Consumption (kWh/100km) 19 non-null float64
6 Battery_Health (%) 19 non-null float64
7 Charging_Type 20 non-null str
dtypes: float64(6), str(2)
memory usage: 1.4 KB
None
Statistical Summary:
Battery Temperature (°C) Charging_Time (hrs) ... Energy_Consumption (kWh/100km) Battery_Health (%)
count 19.000000 19.000000 ...
19.000000 19.000000
mean 40.052632 4.163158 ...
16.468421 87.052632
std 8.316474 3.354137 ...
2.160775 8.947214
min 29.000000 0.700000 ...
13.500000 72.000000
25% 32.500000 1.050000 ...
14.400000 79.500000
50% 36.000000 1.600000 ...
17.000000 91.000000
75% 48.000000 7.350000 ...
18.350000 95.500000
max 52.000000 8.800000 ...
19.500000 98.000000
[8 rows x 6 columns]

Correlation Between EV Battery Features

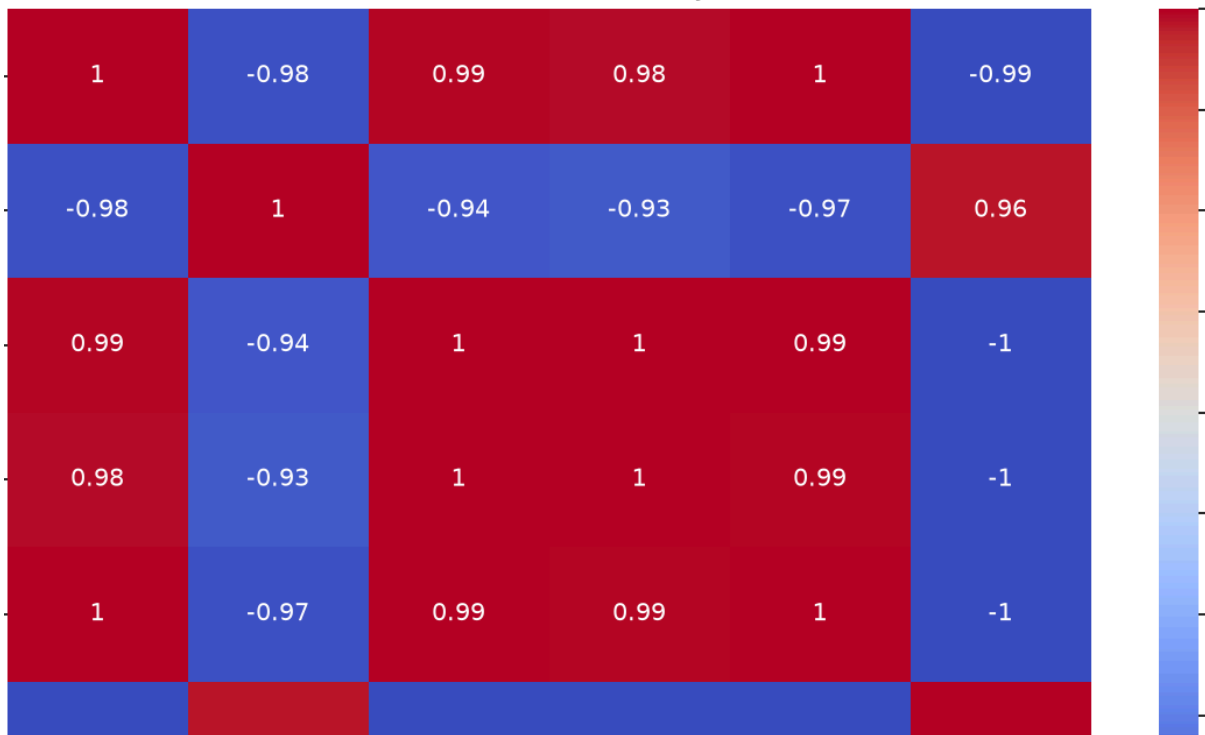


Figure 1

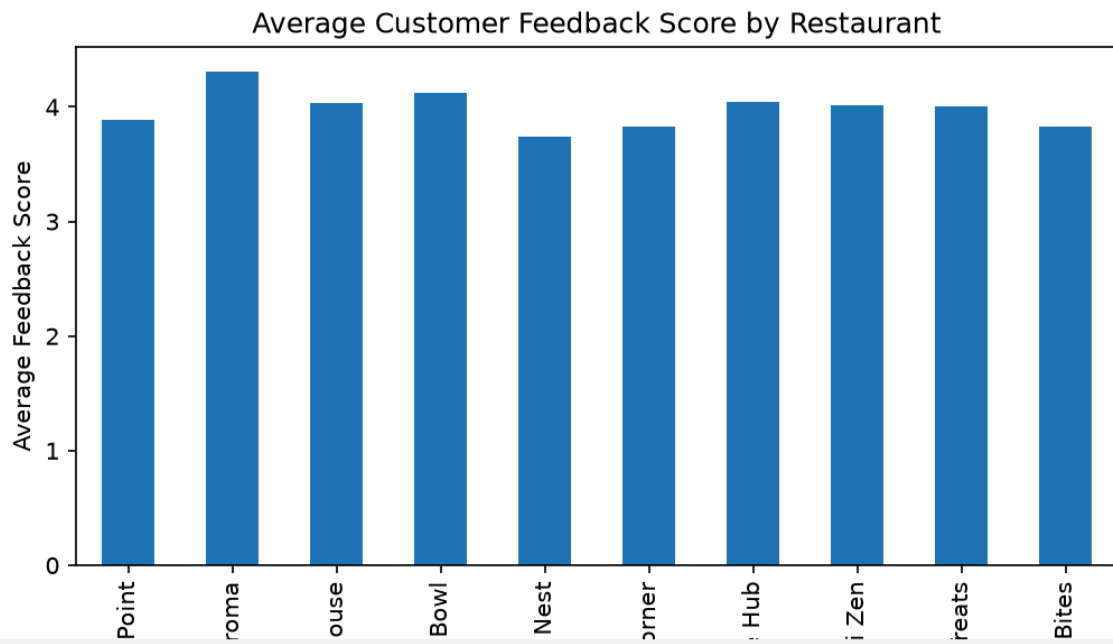


Figure 1

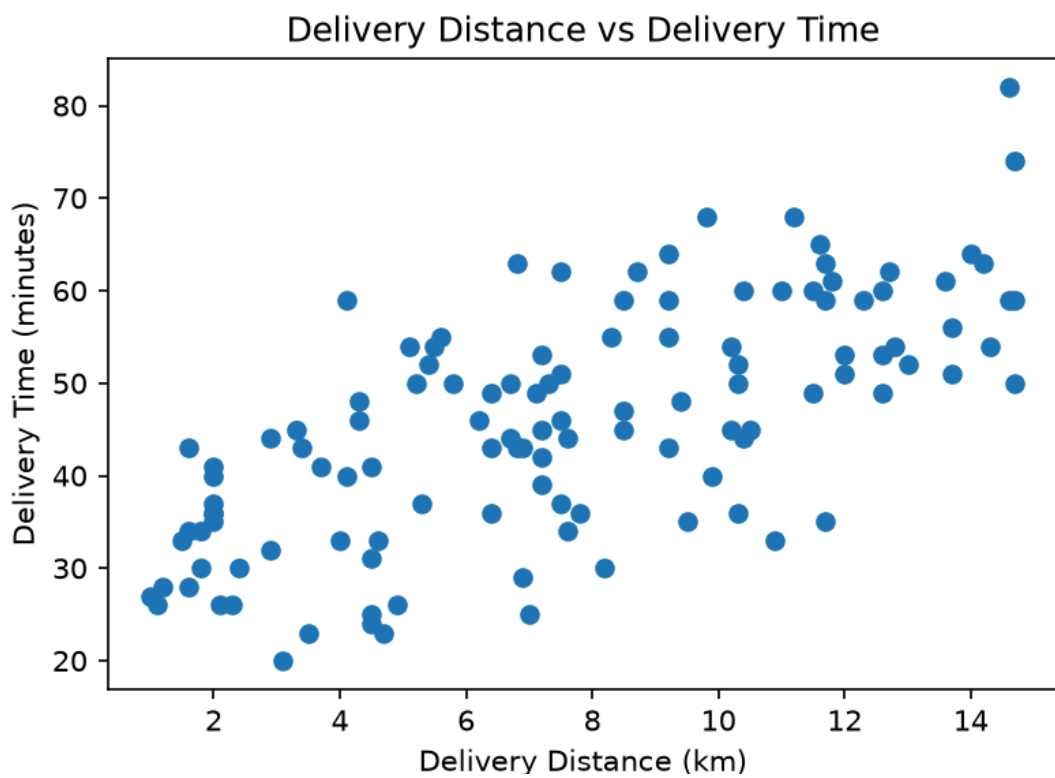


Figure 1

